
Traduction Multilingue avec NLLB-200

Vers un Système de Traduction Médical Pivot

Éwé ↔ Français | Arabe Égyptien ↔ Français

Fine-tuning NLLB-200 · Triangulation Sémantique · Bridge MPNet → ST5

Table des matières

1 Synthèse et Architecture du Système	3
1.1 Vision Globale	3
1.2 Estimation des Ressources	3
1.3 Défis et Enjeux des Données	3
2 Composant ASR (Speech-to-Text)	3
2.1 OmniASR	3
2.1.1 OmniASR pour l'Éwé	4
2.1.1.1 Historique des entraînements	4
2.1.1.2 Architecture du modèle	4
2.1.1.3 Configurations optimales	4
2.1.1.4 Analyse des résultats	4
2.1.2 OmniASR pour l'Arabe Égyptien	4
2.1.2.1 Données d'entraînement	5
2.1.2.2 Hyperparamètres par stage	5
2.1.2.3 Résultats	5
2.2 Whisper pour l'Éwé	5
2.2.1 Données	5
2.2.1.1 Corpus de Parole Ewe	5
2.2.2 Méthodologie	6
2.2.2.1 Pipeline de Prétraitement Audio	6
2.2.2.2 Configuration du Fine-tuning	6
2.2.2.3 Stratégie d'Initialisation du Décodeur	7
2.2.2.4 Métriques d'Évaluation	7
2.2.3 Résultats – RAP Ewe	7
2.2.3.1 Comparaison de Toutes les Configurations	7
2.2.3.2 Dynamique d'Entraînement	8
2.2.3.3 Effet de la Normalisation	8
2.2.3.4 Référence Zéro-Shot	9
2.2.4 Analyse et Discussion	9
2.2.4.1 Pourquoi le Yoruba Accélère la Convergence	9
2.2.4.2 Criticité du Pipeline Audio	9
2.2.4.3 Comparaison avec les Travaux Antérieurs	9
2.2.5 Défis Techniques et Solutions	9
3 Composant LID (Language Identification)	10
3.1 Modèle et Architecture	10
3.2 Résultats	10
4 Composant MT (Machine Translation) : Architecture NLLB-200	10
4.1 Le modèle NLLB-200	10
4.2 Approche Pivot via le Français et Augmentation	10
5 Datasets : Collecte et Architecture Modulaire	11
5.1 Organisation et Sources de Données	11
5.1.1 Corpus Éwé	11
5.1.2 Corpus Arabe Égyptien	11
5.1.3 Langues Sœurs et Augmentation (Fongbe)	11
5.2 Scripts de Téléchargement : Modularité et Portabilité	11
6 Standardisation et Prétraitement	12
6.1 Construction du Pivot Francophone via NLLB-200	12
6.2 Focus : Prétraitement du Dataset Kaggle Q/R	12
6.2.1 Structure du dataset	12
6.2.1.1 Prétraitement	12

7	Expériences de Benchmarking par Backtranslation	13
8	Fine-tuning NLLB pour l'Éwé	13
8.0.0.1	Interprétation :	14
9	Fine-tuning NLLB pour l'Arabe Égyptien	14
9.1	Contexte et Configuration	14
9.2	Résultats	15
9.3	Analyse	15
10	Expérience de Triangulation Sémantique	16
10.1	Motivation et Intuition	16
10.2	Algorithme	16
10.3	Setup Expérimental	17
10.4	Résultats	17
10.5	Analyse et Interprétation	17
10.5.1	Robustesse au masquage	17
10.5.2	Comportement de la courbe moyenne	17
10.5.3	Ce que prouve cette expérience	18
10.6	Limite	18
11	Bridge MPNet → ST5 : Vers un Décodage des Embeddings	18
11.1	Motivation	18
11.2	Pipeline Cible	18
11.3	Architecture du Bridge	18
11.4	Résultats	19
11.5	Conclusion et Perspectives	19
A	Ressources du Projet	19

1. Synthèse et Architecture du Système

1.1 Vision Globale

L'objectif est de réduire les barrières linguistiques lors des consultations médicales via un outil de traduction bidirectionnelle (**Éwé/Arabe Égyptien** ↔ **Français**). Ces langues étant à faibles ressources, l'entraînement des modèles est un défi que nous nous proposons d'attaquer. Ce outil sera mis à disposition des utilisateurs via MedBridge-AI, une application user friendly accessible, dont nous faisons une démonstration lors de la présentation de nos résultats.

Le système repose sur un flux de traitement séquentiel allant de la capture vocale à la synthèse de la réponse. Le tableau ci-dessous récapitule les composants critiques, les modèles sélectionnés et leur rôle au sein du pipeline.

Composant	Modèle / Type & Fonction
ASR	Whisper (Large-v3) / Omnilingual (Google) : Conversion de la parole en texte (Speech-to-Text).
LID	GlottLID : Identification automatique de la langue pour le routage vers les modèles spécifiques.
Traduction (MT)	NLLB-200 (1.3B) : Cœur du système de traduction, fine-tuné sur nos corpus pivot (Français).
TTS	[HAMAD] : Synthèse vocale pour restituer la traduction oralement dans la langue cible.
Datasets	Agrégation de corpus hétérogènes : Audio brut (HD), textes médicaux, et données synthétiques.

TABLE 1 – Récapitulatif des composants et de l'architecture technique

1.2 Estimation des Ressources

Le projet manipule des volumes de données importants. Bien que les modèles de traduction et d'identification soient relativement légers (< 20 GB cumulés), la gestion des datasets (audio brut HD, fichiers de synthèse) ainsi que les checkpoints d'entraînement représentent la part prépondérante des ressources nécessaires.

1.3 Défis et Enjeux des Données

L'implémentation de ce pipeline se heurte à la rareté des ressources de haute qualité pour l'Éwé et l'Arabe Égyptien. Les enjeux qui sous-tendent chaque étape du projet peuvent se résumer ainsi :

- **Langues à faibles ressources** : manque structurel de corpus (audio-texte pour l'ASR, parallèle pour la MT) limitant l'entraînement direct.
- **Spécificité du domaine médical** : absence de datasets cliniques spécialisés pour ces paires linguistiques.
- **Exigence de fidélité clinique** : Qu'il s'agisse de la transcription initiale ou de la synthèse finale, la précision est essentielle pour garantir la sécurité des échanges en contexte de consultation.

Dans les sections suivantes, nous détaillerons les jeux de données spécifiquement mobilisés et les stratégies de prétraitement adoptées pour chaque composant du pipeline (ASR, MT, TTS), tout en gardant à l'esprit que ces défis de rareté et de précision dictent l'ensemble de nos choix techniques.

2. Composant ASR (Speech-to-Text)

Le composant de reconnaissance automatique de la parole (ASR) constitue la première étape du pipeline MedBridge-AI. Deux architectures ont été explorées — **omniASR_CTC_300M** (Meta) et **Whisper** (OpenAI) — chacune appliquée à l'Éwé et à l'arabe égyptien.

2.1 OmniASR

2.1.1 OmniASR pour l'Éwé

Le fine-tuning du modèle **omniASR_CTC_300M** (Meta, 325 millions de paramètres), basé sur une architecture wav2vec2, a été appliqué à l'Éwé, une langue à faibles ressources. L'objectif est d'obtenir un système robuste dans un contexte médical malgré des contraintes importantes de volume de données et d'hétérogénéité orthographique.

2.1.1.1 Historique des entraînements

Le développement du modèle s'est appuyé sur une démarche itérative structurée, permettant d'identifier progressivement les facteurs limitants et les configurations optimales.

Version	Dataset	Val WER (%)	Test WER brut (%)
V1	parquet_omni	72.7	–
V2	parquet_omni	36.5	64.5
V3	parquet_omni	35.9	63.8
V4	parquet_omni	34.0	63.4
V5	parquet_omni_clean	34.9	64.5
V6	parquet_omni_v6	–	63.9
V6_clean	parquet_omni_v6_clean	34.9	63.2

TABLE 2 – Évolution des performances du modèle OmniASR — Éwé

La version V1 a révélé un problème critique : l'encodeur restait gelé pendant tout l'entraînement, empêchant toute adaptation réelle. À partir de V2, le dégel de l'encodeur (step 1000) a permis une amélioration significative des performances. Les versions suivantes ont exploré la réduction du batch effectif (V3–V4), le nettoyage des données (V5), une refonte complète du pipeline de construction des données (V6), et une décontamination stricte des splits (V6_clean). La version finale **V6_clean** constitue le meilleur compromis obtenu.

2.1.1.2 Architecture du modèle

Le modèle **omniASR_CTC_300M** est basé sur une architecture wav2vec2 adaptée au multilinguisme. Il se compose de trois modules principaux : un encodeur acoustique (extraction de représentations à partir du signal audio brut à 16 kHz), un encodeur Transformer (modélisation des dépendances temporelles via self-attention), et une tête CTC (projection vers un vocabulaire de caractères). Le fine-tuning est réalisé en deux phases : encodeur gelé dans un premier temps pour adapter la tête CTC, puis dégel complet. Le choix du CTC s'explique par sa robustesse dans des contextes à faibles ressources, nécessitant moins de données que les approches auto-régressives.

2.1.1.3 Configurations optimales

La configuration optimale correspond à la version V6_clean : learning rate 1×10^{-5} , freeze encoder 1000 steps, gradient accumulation 2 (batch effectif ~ 92), max elements 1.92M, activation checkpointing layerwise, mixed precision bfloat16.

2.1.1.4 Analyse des résultats

Métrique	WER (%)	CER (%)
Brut	63.20	19.96
Normalisation standard	34.06	9.86
Normalisation Éwé	33.35	9.55

TABLE 3 – Impact de la normalisation sur les performances — OmniASR Éwé

L'écart entre validation ($\sim 35\%$) et test brut ($\sim 63\%$) est principalement dû à des incohérences orthographiques et Unicode : variantes de caractères (ex : $\varepsilon \leftrightarrow e$), ponctuation, espacement. Après normalisation spécifique à l'Éwé, cet écart disparaît quasiment et le WER atteint 33.35%, estimation plus fidèle de la performance réelle.

2.1.2 OmniASR pour l'Arabe Égyptien

L'entraînement du modèle **omniASR_CTC_300M_v2** pour l'arabe égyptien suit une approche en trois stages progressifs, du général vers le spécifique.

2.1.2.1 Données d'entraînement

Corpus	Type	Utterances	Durée	Stage
MGB3	Arabe égyptien général	2 153	~3h	Stage 1
NileTTS	Arabe égyptien synthétique	8 571	38h	Stage 2
NileTTS Medical	Terminologie médicale	1 411	~6h	Stage 3

TABLE 4 – Corpus utilisés pour le fine-tuning OmniASR — Arabe Égyptien

2.1.2.2 Hyperparamètres par stage

Paramètre	Stage 1	Stage 2	Stage 3
Learning Rate	3×10^{-6}	5×10^{-7}	1×10^{-7}
Steps	3 000	5 000	3 000
Batch size effectif	8	8	8
Warmup steps	200	200	100
Scheduler	Cosine + warmup		
Optimiseur	AdamW, weight decay 0.01		
Précision	BFloat16		

TABLE 5 – Hyperparamètres par stage — OmniASR Arabe Égyptien

2.1.2.3 Résultats

Stage	Dataset	WER (%)	CER (%)
Stage 1	MGB3	60.8	–
Stage 2	NileTTS	45.12	16.32
Stage 3 (final)	NileTTS Medical	33.62	9.51

TABLE 6 – Évolution des performances par stage — OmniASR Arabe Égyptien

Le WER passe de 60.8% à 33.62% entre le Stage 1 et le Stage 3, soit une réduction de 44.5%. Le CER final de 9.51% indique que 90 caractères sur 100 sont correctement reconnus. Le nombre de steps du Stage 2 (5 000 steps $\approx$ 0.58 epoch sur NileTTS) est délibérément limité pour éviter le surapprentissage sur un corpus de taille modérée.

2.2 Whisper pour l'Éwé

2.2.1 Données

2.2.1.1 Corpus de Parole Ewe

Notre jeu de données provient du corpus SpeechUG et Waxal (google) distribué au format HuggingFace Parquet. Chaque entrée contient les octets bruts d'un fichier MP3 et sa transcription textuelle. Après fusion de deux fichiers sources, déduplication et suppression des chevauchements inter-splits, le Tableau 7 présente les statistiques finales.

TABLE 7 – Répartition des données après nettoyage.

Split	Exemples	Chevauchement
Train (total)	30 642	–
Validation	1 614	0
Test (propre)	1 443	0
Doublons supprimés	1 429	–

Deux fichiers sources ont été fusionnés : `ewe-train-00*.parquet` (15 053 exemples) et `ewe_train.parquet` (15 589 exemples uniques après déduplication). Un audit strict a garanti zéro chevauchement entre les splits train, validation et test, assurant une évaluation honnête sans fuite de données (*data leakage*).

2.2.2 Méthodologie

2.2.2.1 Pipeline de Prétraitement Audio

L’une des découvertes majeures de ce travail est la *dépendance critique* des performances du modèle à la bibliothèque de décodage MP3 utilisée. L’extracteur de caractéristiques de Whisper est sensible à la représentation exacte de la forme d’onde : différents décodeurs (libav, ffmpeg, audioread) introduisent des différences infra-échantillon amplifiées par le calcul du mel-spectrogramme, créant une inadéquation de distribution entre l’entraînement et l’inférence.

Nous avons déterminé que le pipeline suivant, utilisé de manière cohérente à l’entraînement, à la validation et à l’inférence, produit un WER de 17,15 % :

Listing 1 – Pipeline de chargement audio (critique).

```
import io, librosa

def charger_audio(octets_mp3, sr=16000):
    forme_onde, _ = librosa.load(
        io.BytesIO(octets_mp3),
        sr=sr, mono=True
    )
    # Suppression des silences
    forme_onde, _ = librosa.effects.trim(
        forme_onde, top_db=20
    )
    return forme_onde
```

Le Tableau 8 quantifie l’impact de pipelines alternatifs sur le *même* modèle fine-tuné.

TABLE 8 – Impact du pipeline MP3 sur le WER (mêmes poids de modèle).

Pipeline audio	WER
<code>librosa.load(BytesIO)</code> – <i>pipeline entr.</i>	17,15 %
<code>ffmpeg</code> → WAV → <code>librosa</code>	98,59 %
Chargeur Audio() HuggingFace	98,90 %

2.2.2.2 Configuration du Fine-tuning

Toutes les expériences utilisent le point de contrôle pré-entraîné `openai/whisper-medium`. La configuration d’entraînement est détaillée dans le Tableau 9.

TABLE 9 – Hyperparamètres du fine-tuning.

Hyperparamètre	Valeur
Modèle de base	openai/whisper-medium (307M)
Learning rate	1×10^{-5}
Planif. LR	Warmup lin. + déc. cosinus
Warmup steps	200
Batch size eff.	16 (8×2 accum. grad.)
Epochs	5
Précision	FP16 (mixte)
Grad. checkpointing	Activé
Métrique	WER (meilleur modèle sauvegardé)
GPU	NVIDIA A100 40 Go (helios, SLURM)
Durée entr.	≈ 32 heures

2.2.2.3 Stratégie d’Initialisation du Décodeur

Pour forcer le décodage en Yoruba, nous procédons comme suit :

Listing 2 – Initialisation des tokens Yoruba (méthode correcte).

```
# CRITIQUE : définir dans model.config uniquement
# Ne PAS passer forced_decoder_ids à generate()
# en meme temps -- cela cause un conflit ValueError
model.config.forced_decoder_ids = None
model.config.suppress_tokens = []
ids_forces = processor.get_decoder_prompt_ids(
    language="yo", task="transcribe"
)
model.config.forced_decoder_ids = ids_forces
```

Passer `forced_decoder_ids` simultanément dans `model.config` et dans `generate()` déclenche une `ValueError` due à un `ForceTokensLogitsProcessor` dupliqué.

2.2.2.4 Métriques d’Évaluation

Nous rapportons les résultats à trois niveaux de normalisation :

1. **WER/CER bruts** : aucune normalisation du texte.
2. **Normalisation standard** : normalisation Unicode NFKC, suppression de la ponctuation, mise en minuscules.
3. **Normalisation spécifique Ewe** : normalisation standard plus correspondances phonétiques (o ouvert $\rightarrow$ o, e ouvert $\rightarrow$ e, ng $\rightarrow$ n, etc.).

La normalisation Ewe est motivée par les incohérences orthographiques du corpus : certains annotateurs substituent des caractères ASCII standard aux équivalents diacritiques.

2.2.3 Résultats – RAP Ewe

2.2.3.1 Comparaison de Toutes les Configurations

Le Tableau 10 présente les résultats complets en WER et CER pour toutes les configurations testées.

TABLE 10 – Résultats WER et CER sur le jeu de test Ewe (1 443 exemples). Brut = sans normalisation ; Norm. = normalisation spécifique Ewe. ★ indique le résultat officiel du script d’entraînement.

Modèle	Décodeur	Données train	WER _{brut}	CER _{brut}	WER _{norm}	CER _{norm}	Gain vs SOTA
Whisper Medium (zéro-shot)	lang:en	–	≈210 %	≈85 %	–	–	–
Whisper Medium (zéro-shot)	lang:yo	–	114,65 %	71,29 %	111,39 %	66,13 %	–
Whisper Base (Dei et al., 2025)	–	107h	37,00 %	12,00 %	–	–	référence
Fine-tuné V1	lang:en	15 053 ex.	29,53 %	–	–	–	–7,47 pp
Fine-tuné V2	lang:en	29 362 ex.	27,42 %	–	–	–	–9,58 pp
Pipeline ffmpeg	lang:en	29 362 ex.	98,90 %	–	–	–	–
Fine-tuné Yoruba★	lang:yo	30 642 ex.	17,15 %	6,14 %	14,21 %	4,79 %	–19,85 pp

2.2.3.2 Dynamique d’Entraînement

La Figure 1 présente l’évolution du WER de validation au cours des epochs pour le modèle initialisé avec le Yoruba.

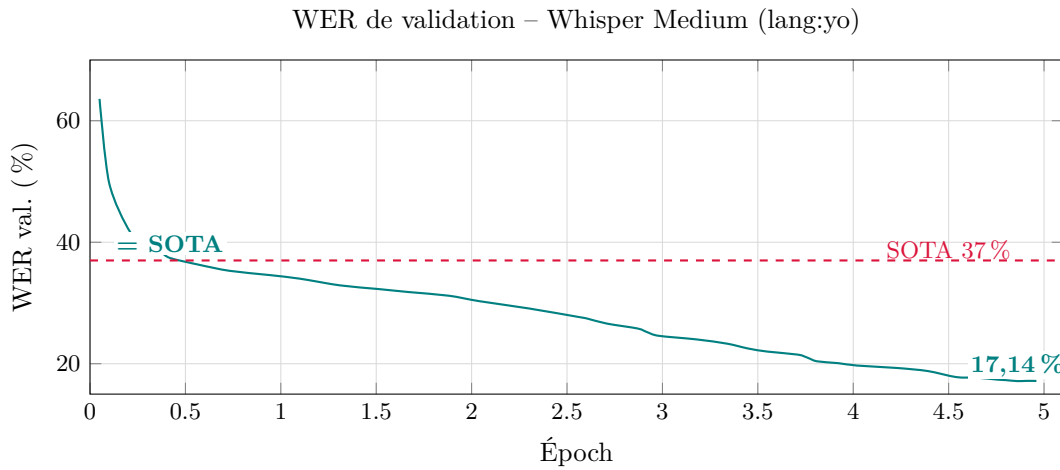


FIGURE 1 – WER de validation par epoch (modèle lang :yo). Le modèle atteint l’état de l’art de 37 % dès l’epoch 0,42 et son meilleur checkpoint (17,14 %) à l’epoch 4,86.

Trois jalons clés illustrent l’efficacité du Yoruba : (i) **epoch 0,05** : WER 63,6 % dès le premier éval (vs. 100 % avec lang:en); (ii) **epoch 0,42** : l’état de l’art de Dei et al. est égalé après seulement une demi-epoch; (iii) **epoch 4,86** : meilleur WER de validation de 17,14 %.

2.2.3.3 Effet de la Normalisation

Le Tableau 11 quantifie la contribution de la normalisation textuelle au WER rapporté.

TABLE 11 – Effet de la normalisation sur le meilleur modèle (lang :yo, 1 443 ex.).

Niveau de normalisation	WER	CER
Aucune (brut)	17,15 %	6,14 %
Standard	≈15,5 %	≈5,2 %
Spécifique Ewe	14,21 %	4,79 %
<i>Gain (brut → Ewe)</i>	–2,94 pp	–1,35 pp

Le CER_{brut} de 6,14 % – bien inférieur au WER_{brut} de 17,15 % – confirme que le modèle prédit correctement la grande majorité des caractères, les erreurs restantes étant concentrées sur les diacritiques et les frontières de mots.

2.2.3.4 Référence Zéro-Shot

Pour référence, le Tableau 10 inclut les performances zéro-shot de Whisper Medium non fine-tuné avec `lang:en` ($\approx 210\%$ WER) et `lang:yo` ($114,65\%$ WER). La référence Yoruba zéro-shot est substantiellement meilleure ($-95,35$ points absolus), confirmant que l’initialisation par une langue africaine tonale constitue un point de départ bien plus favorable.

2.2.4 Analyse et Discussion

2.2.4.1 Pourquoi le Yoruba Accélère la Convergence

Notre hypothèse est que l’initialisation via le Yoruba bénéficie à la RAP Ewe par deux mécanismes complémentaires :

(1) Structure prosodique partagée. L’Ewe et le Yoruba sont des langues tonales d’Afrique de l’Ouest. L’encodeur Whisper, pré-entraîné sur 680 000 heures incluant des données Yoruba, capture déjà des représentations acoustiques adaptées à la parole tonale ouest-africaine. Cela fournit un point de départ nettement meilleur que les représentations anglaises.

(2) Distribution des tokens plus proche. Bien que les points de code Unicode diffèrent (*ex.* O ponctuel Yoruba vs. o ouvert Ewe), la distance phonétique entre les tokens Yoruba et Ewe est inférieure à la distance par rapport aux tokens anglais. Le décodeur produit donc moins de “tokens indéterminés” lors des premières étapes d’entraînement, permettant à la perte de décroître dès l’époch 0.

2.2.4.2 Criticité du Pipeline Audio

La sensibilité à la bibliothèque de décodage MP3 est une découverte pratique aux importantes implications pour la reproductibilité. Le front-end mel-spectrogramme de Whisper amplifie les différences infra-échantillon introduites par différents décodeurs MP3, créant un décalage de distribution entre les caractéristiques d’entraînement et d’inférence.

Cette observation implique que :

- Les fiches techniques (*model cards*) des modèles fine-tunés Whisper devraient explicitement spécifier la bibliothèque de chargement audio et sa version.
- Les scripts d’évaluation devraient être distribués avec les poids du modèle pour assurer des mesures de WER reproductibles.

2.2.4.3 Comparaison avec les Travaux Antérieurs

Seul le travail de Dei et al. (2025) publie un résultat sur la RAP Ewe : WER 37% avec Whisper Base sur 107 heures annotées. Notre résultat de $17,15\%$ représente une réduction relative de 54% . Cette amélioration est attribuable à trois facteurs : (i) un modèle plus grand (Medium vs. Base, 307M vs. 74M paramètres), (ii) davantage de données d’entraînement (30 642 vs. non spécifié), et (iii) la stratégie d’initialisation via le Yoruba.

2.2.5 Défis Techniques et Solutions

Le Tableau 12 récapitule les principaux problèmes techniques rencontrés et leurs résolutions. Nous les reportons dans un souci de reproductibilité, car des problèmes similaires peuvent survenir lors du fine-tuning de Whisper sur des langues à faibles ressources.

TABLE 12 – Défis techniques et solutions apportées.

Problème	Symptôme	Solution
Pipeline audio	WER 98,90 % avec ffmpeg	librosa BytesIO cohérent
Tokenizer anglais	WER bloqué à 100 %	Passage à <code>lang:yo</code>
ForceTokens double	ValueError generate()	Ids forcés dans <code>model.config</code> unique- ment
Quota NFS dépassé	Crash à l’étape 6 400	<code>HF_HOME + save_total_limit=2</code>
ZeroDivisionError	WER crashe	Exposer exceptions ; vérifier preds/refs vides
<code>lang:yo</code> non forcé in- férence	WER 99 % modèle FT	Forcer <code>forced_decoder_ids</code> à l’infé- rence
Chevauchement splits	WER surestime performances	Suppression 1 429 doublons

3. Composant LID (Language Identification)

L'étape de LID est cruciale pour router le texte vers le bon modèle de traduction.

3.1 Modèle et Architecture

GlottLID est un modèle de classification basé sur fastText, développé par l'Université de Munich. Il supporte plus de 1 600 langues et dialectes, dont plusieurs variantes de l'arabe : égyptien (`arz_Arab`), standard (`arb_Arab`), levantin (`apc_Arab`), entre autres. Sa légèreté le rend adapté à une intégration dans un pipeline temps réel.

3.2 Résultats

Variante	Code	Précision
Arabe standard	<code>arb_Arab</code>	jusqu'à 99%
Arabe égyptien	<code>arz_Arab</code>	39% à 95%

TABLE 13 – Performances de GlottLID sur les variantes arabes testées

La précision sur l'arabe égyptien varie entre 39% et 95% selon la clarté de l'audio en entrée — ce qui reflète la difficulté inhérente à distinguer les dialectes arabes proches, notamment lorsque le signal est bruité ou la diction peu marquée. L'identification correcte de la langue a été confirmée sur l'ensemble des tests réels effectués dans le pipeline.

4. Composant MT (Machine Translation) : Architecture NLLB-200

4.1 Le modèle NLLB-200

Pour la traduction automatique nous avons choisi le modèle **NLLB-200** (No Language Left Behind) développée par Meta AI, qui a été conçue spécifiquement pour adresser le fossé numérique entre les langues à hautes ressources et celles historiquement délaissées par la recherche en NLP, comme l'Éwé.

NLLB-200 repose sur une architecture de type **Transformer Encodeur-Décodeur** massivement multilingue. Contrairement aux modèles traditionnels qui nécessitent des paires de langues spécifiques, NLLB utilise un espace de représentation partagé capable de traiter plus de 200 langues simultanément.

Les caractéristiques clés de cette architecture pour notre projet sont :

- **Transfert de connaissances (Transfer Learning)** : En apprenant à traduire des centaines de langues en même temps, le modèle développe des capacités de compréhension sémantique qui bénéficient aux langues à faibles ressources par analogie avec des langues plus documentées.
- **Gestion des identifiants de langue** : Chaque séquence d'entrée est préfixée par un jeton de langue source (*source language token*) et le décodeur est amorcé avec un jeton de langue cible (*target language token*). Ces jetons permettent le contrôle de la langue générée par le modèle.

4.2 Approche Pivot via le Français et Augmentation

Face à l'absence de corpus parallèles Éwé-Arabe directs, nous adoptons une stratégie pivot utilisant le **Français comme langue intermédiaire**. Pour pallier le manque de ressources, d'autres langues ont été mobilisées :

- **Anglais** : Augmentation de données via les corpus anglais-éwé et anglais-arabe existants.
- **Fongbe** : Utilisation de cette langue sœur de l'Éwé pour enrichir les représentations par co-apprentissage.

5. Datasets : Collecte et Architecture Modulaire

5.1 Organisation et Sources de Données

Le projet repose sur l'agrégation de corpus multilingues provenant de plus de dix sources académiques et publiques. Afin de garantir la reproductibilité, les données sont organisées de manière modulaire par langue cible dans une structure hiérarchique standardisée (`data/data_[langue]`).

5.1.1 Corpus Éwé

L'Éwé constitue l'un des piliers du projet avec environ 240 000 lignes de données brutes collectées, incluant des ressources textuelles et audio.

Source (Répertoire)	Description et Usage
AI4D_MT	Corpus parallèle Français-Éwé (~23 662 paires) issu de Zenodo.
MAFAND-MT	Corpus de news (Masakhane) parallèle Français-Éwé (~5 003 paires).
ewe_anglais	Alignements Anglais-Éwé translittérés en Français-Éwé (~491 232 paires) pour l'augmentation de données par traduction automatique de l'anglais vers le français.

TABLE 14 – Inventaire des ressources pour la langue Éwé

5.1.2 Corpus Arabe Égyptien

Les données pour l'arabe égyptien se concentrent sur le dialogue et le domaine médical (via NileTTS), complétées par des corpus littéraires pour la diversité stylistique.

Source (Répertoire)	Description et Usage
Kaggle	Corpus parallèle multi-domaine (~37 149 paires) Anglais-Arabe égyptien aligné.
Mendeley	Corpus parallèle de textes littéraires (ex : "Le Petit Prince") (~1 481 paires) pour l'alignement de précision.
Egyptian_Dialogue	Corpus parallèle conversationnel (~4 322 paires) capturant les nuances du dialecte égyptien.

TABLE 15 – Inventaire des ressources pour l'Arabe Égyptien

5.1.3 Langues Sœurs et Augmentation (Fongbe)

Le Fongbe est intégré comme langue pivot et de co-apprentissage en raison de sa proximité structurelle avec l'Éwé.

Source (Répertoire)	Description et Usage
FFR_v1	Corpus Fongbe-Français (~53 975 paires) avec gestion fine des diacritiques.
AI4D	Corpus Fongbe-Français (~53 366 paires) issu du projet AI4D.
MAFAND	Corpus Fongbe-Français additionnel (~5 443 paires) de news (Masakhane).

TABLE 16 – Ressources Fongbe pour le transfert linguistique

5.2 Scripts de Téléchargement : Modularité et Portabilité

Le projet repose sur une méthode de téléchargement des données automatisée : chaque dataset est téléchargé en exécutant un script python dédié (dans `data/download_scripts/`). Cette approche modulaire permet d'ajouter de nouveaux datasets en assurant la traçabilité et sans qu'il y ait de mélange ou fuite avec d'autres datasets.

Chaque script suit une **architecture commune** :

- **Entrée flexible** : APIs (HuggingFace, Zenodo REST), GitHub raw content (JSONL/CSV), fichiers locaux (Excel, parquet).
- **Sauvegarde locale** : chaque source obtient son dossier isolé ; réexécuter le script = re-télécharger les données.
- **Modularité** : ajouter une nouvelle source = créer un nouveau script sans modifier l'existant.
- **Portabilité** : les scripts fonctionnent sur toute machine avec Python + internet.

- **Traçabilité** : chaque donnée porte l'étiquette de sa source (`database_origin`).
- **Parallélisation** : les scripts peuvent s'exécuter indépendamment via SLURM.

6. Standardisation et Prétraitement

Après téléchargement, chaque fichier est nettoyé et sauvegardé dans un dossier `standardized` avec une structure de colonnes fixe commune pour toutes les langues. Cela permet de manipuler toutes les sources avec les mêmes outils d'entraînement et d'évaluation.

Colonne	Contenu et Rôle
<code>database_origin</code>	Nom de la source d'origine pour assurer la traçabilité.
<code>id</code>	Identifiant unique (préfixé par la source pour éviter les doublons).
<code>text_[langue]</code>	Texte dans la langue cible (Éwé, Arabe ou Fongbe).
<code>text_french</code>	Traduction française correspondante.

TABLE 17 – Structure standardisée appliquée à tous les datasets

6.1 Construction du Pivot Francophone via NLLB-200

Pour l'Éwé, même si des ressources en français existent, les plus importantes étaient en anglais ; quant à l'Arabe Égyptien, pour ce dernier, aucune ressource en français n'était disponible. Afin de construire notre corpus pivot, nous avons généré la traduction française en utilisant le modèle **NLLB-200-1.3B**, puis standardisé les dataset augmenté dans le format décrit plus haut pour les insérer dans le pipeline de finetuning.

Cette étape de génération de données synthétiques a été automatisée via deux scripts principaux :

- `build_ewe_french_corpus.py` : Conversion des paires Éwé-Anglais en paires `text_ewe` / `text_french`.
- `add_french_translation.py` : Pour l'Arabe Égyptien, standardisation des sources existantes puis génération de la version française pour compléter l'alignement.

6.2 Focus : Prétraitement du Dataset Kaggle Q/R

6.2.1 Structure du dataset

Le dataset Kaggle repose sur un format Q/R séquentiel : sur une même colonne, l'ordre est le suivant : question en anglais, puis en arabe standard, puis en arabe égyptien. Suit la réponse en anglais, puis en arabe standard, puis en arabe égyptien. Les données sont ainsi organisées blocs Q/R, chaque bloc comportant trois lignes.

6.2.1.1 Prétraitement

Le nettoyage du dataset a consisté à isoler les données pertinentes et à assurer l'intégrité des paires linguistiques :

- **Filtrage** : Suppression de l'arabe standard (`ar`) pour ne conserver que les variantes cibles.
- **Validation des séquences** : l'intégrité des données a été garantie en s'assurant du respect de l'ordre de la sequence en -> `ar_egyptien` dans les données restantes. Toutes ligne orpheline (`en` non suivi par `ar_egyptien` ou `ar_egyptien` non précédé par `en` a été supprimées).

Ensuite, les données ont été transformées comme suit : elles ont été **groupées par paire Q/A**, puis transposées par blocs de deux, de manière à ce qu'une même ligne contienne les versions anglaise et arabe égyptien de la même phrase. Cela a permis de standardiser le format pour l'intégrer dans le pipeline de fine-tuning.

Nous attirons l'attention sur l'importance de ce groupement par paire Q/R. Il permet d'éviter que, lors du découpage des données, une question appartenant à l'ensemble d'entraînement ne voie pas la réponse associée basculer dans l'ensemble de validation ou de test. La question et la réponse étant sémantiquement très liées, cette situation constituerait une fuite sémantique. Nous exposerons dans la suite de ce document comment ce problème, non identifié lors de la première expérience de fine-tuning, avait produit des scores BLEU aberrants lors des évaluations.

7. Expériences de Benchmarking par Backtranslation

Une première tentative d'évaluation de la qualité de traduction du modèle NLLB a consisté à réaliser des tests de backtranslation (Source → Français → Source). Si les scores obtenus (Tableau 18) semblent de prime abord acceptables, une analyse comparative avec des paires de test réelles (humaines) nous a conduits à invalider cette méthode de validation.

Paire	BLEU	chrF	Échantillons	Durée
Éwé → Fr → Éwé	11.01	32.30	49	~57.6 s
Arabe → Fr → Arabe	12.18	44.76	950	822.19 s

TABLE 18 – Résultats de backtranslation (modèle NLLB-200 zero-shot)

Analyse critique : Nous avons constaté que les scores BLEU issus de la backtranslation sont environ deux fois supérieurs à ceux obtenus sur de véritables corpus de référence (voir section suivante). Ce phénomène s'explique par une forme de "biais interne" au modèle : même si la traduction intermédiaire en français est de faible qualité, elle conserve une structure sémantique et des biais spécifiques au modèle NLLB. Il est alors plus facile pour le modèle de "revenir" à la phrase source initiale.

En effet, la backtranslation évalue ici la cohérence interne du modèle plutôt que sa fidélité linguistique réelle. Cette approche constitue une **"erreur dans l'erreur"** et ne peut donc pas servir pour évaluer la qualité de la traduction.

8. Fine-tuning NLLB pour l'Éwé

Quatre configurations d'entraînement ont été testées pour la paire Éwé ↔ Français. On distingue deux types de corpus : le corpus *organique*, composé de paires Éwé-Français issues d'un alignement humain préexistant, et le corpus *synthétique*, également constitué de paires Éwé-Français alignées, mais dont la traduction française a été générée automatiquement par NLLB à partir de l'anglais.

1. Fine-tuning avec uniquement du corpus Éwé organique
2. Fine-tuning avec Éwé + Fongbe
3. Fine-tuning Éwé organique + paires Éwé-Fr synthétiques (ratio 1 :2)
4. Fine-tuning Éwé organique + paires Éwé-Fr synthétiques (ratio 1 :4)

Afin de garantir la validité et la comparabilité des résultats, le jeu de validation et le jeu de test sont identiques pour l'ensemble des quatre configurations. Seul le corpus d'entraînement varie d'une configuration à l'autre.

En plus de cela, les quatre configurations partagent les hyperparamètres suivants.

Paramètre	Valeur
Modèle de base	NLLB-200 (1.3B parameters)
Taux d'apprentissage	2×10^{-5}
Taille de lot	16
Weight decay	0,01
Gradient accumulation steps	1
Longueur maximale	128 jetons
Graine aléatoire (<i>seed</i>)	42
Époques maximales	10
Patience (<i>Early Stopping</i>)	2 époques
Métrique de sélection	BLEU (validation)
Précision	fp16
Stratégie d'évaluation	par époque

TABLE 19 – Hyperparamètres communs aux quatre configurations d'entraînement.

Époque	Config. 1 (Baseline)		Config. 2 (+ Fongbe)		Config. 3 (Synthétique 1 :2)		Config. 4 (Synthétique 1 :4)	
	Val	Test	Val	Test	Val	Test	Val	Test
0 (initial)	4,62	27,10	4,62	27,10	4,62	27,10	4,62	27,10
1	15,29	—	12,52	—	13,36	—	16,68	—
2	15,82	—	12,02	—	15,88	—	13,30	—
3	16,17	—	16,35	—	12,42	—	12,95	24,92
4	16,34	—	23,33	—	16,62	—	—	—
5	16,05	—	17,57	—	13,17	—	—	—
6 (final)	16,34	18,21	16,33	11,85	13,40	16,82	—	—

TABLE 20 – Évolution des scores BLEU (Val / Test) par époque pour les quatre configurations.

8.0.0.1 Interprétation :

Avant tout fine-tuning, le score BLEU sur le jeu de test est anormalement élevé (27,10) alors que celui sur la validation est très bas (4,62). Cet écart s'explique très probablement par un artefact de partition : le jeu de test contient par hasard des phrases proches de celles sur lesquelles NLLB avait été entraîné chez Meta.

Au fil des époques, la validation monte et le test descend — ce qui est le comportement attendu : le modèle arrête de profiter de cet avantage initial et apprend à traduire de façon plus uniforme sur l'ensemble du corpus.

La configuration Fongbe (Config. 2) est mise de côté : le score de test final (11,85) est le plus bas de toutes les configurations, ce qui montre que l'ajout de données d'une langue voisine n'aide pas et dégrade même les résultats.

Pour les configurations restantes, on calcule la moyenne Val/Test finale, les deux jeux ayant le même nombre de phrases :

Configuration	BLEU Val	BLEU Test	Moyenne
One-shot (sans fine-tuning)	4,62	27,10	15,86
Config. 1 — Baseline	16,34	18,21	17,28
Config. 3 — Synthétique 1 :2	13,40	16,82	15,11
Config. 4 — Synthétique 1 :4	12,95	24,92	18,93

TABLE 21 – Scores BLEU finaux et moyenne Val/Test pour les quatre configurations retenues.

La Config. 4 a la moyenne la plus haute (18,93), mais l'entraînement s'est arrêté à l'époque 3 seulement. Le score de test (24,92) est encore gonflé par l'artefact initial — si l'entraînement avait continué, il aurait probablement continué à baisser comme dans les autres configurations. L'early stopping s'est déclenché tôt parce que le volume de données synthétiques a rapidement fait baisser le BLEU sur le set de validation, ce qui a produit un arrêt à un moment favorable mais pas représentatif.

Le one-shot affiche une moyenne de 15,86, proche de la Config. 3 (15,11), mais pour une raison opposée : un test artificiellement haut contre une validation très basse. Ce n'est pas une performance réelle.

La Config. 1 reste la référence la plus solide : moyenne de 17,28, scores Val et Test proches et stables sur six époques. Ces résultats indiquent que la qualité des données d'entraînement compte plus que leur volume : ni les données synthétiques ni les données multilingues n'ont amélioré les résultats par rapport au corpus organique seul.

9. Fine-tuning NLLB pour l'Arabe Égyptien

9.1 Contexte et Configuration

Les expériences sur l'arabe égyptien utilisent un corpus entièrement synthétique, faute de données parallèles authentiques pour cette langue à ressources limitées. Deux expériences ont été menées : une première affectée par une fuite sémantique dans le découpage des données (décrite en détail sections 6.2 et 9.2), et une seconde corrigée servant de référence.

Les détails techniques (modèle, choix des hyperparamètres) de la configuration communes aux deux expériences sont listés dans le tableau suivant :

Paramètre	Valeur
Modèle de base	NLLB-200 (1.3B parameters)
Dataset	<code>unified_arabic_master_corpus.csv</code>
Taille après nettoyage	24 447 paires (185 lignes supprimées)
Répartition	Train=22 000 / Val=1 227 / Test=1 220
Taux d'apprentissage	2×10^{-5}
Taille de lot	16
Longueur maximale	128 jetons
Époques maximales	10
Patience (<i>Early Stopping</i>)	2 époques
Métrique de sélection	BLEU (sacrebleu)

TABLE 22 – Hyperparamètres communs aux deux expériences — Arabe Égyptien.

9.2 Résultats

Les scores BLEU ci-dessous correspondent au jeu de validation. C'est leur comportement d'une époque à l'autre qui est instructif : en v1, les valeurs sont erratiques, signe que la métrique est corrompue par la fuite sémantique ; en v2, le score se stabilise dès l'époque 2 et ne bouge plus, ce qui reflète une convergence réelle du modèle.

Époque	Expérience v1 (Fuite sémantique)		Expérience v2 (Corrigée)	
	BLEU	Loss	BLEU	Loss
0 (initial)	5,30	1,575	5,30	1,575
1	60,43	0,983	19,34	0,984
2	51,18	0,949	20,45	0,955
3	71,69	0,950	20,45	0,954
4 (final)	71,69	0,963	20,45	0,973
5 (stop v1)	57,42	—	—	—

TABLE 23 – Évolution des scores BLEU et de la loss par époque pour les deux expériences — Arabe Égyptien.

9.3 Analyse

Le paradoxe central de l'expérience v1 est le suivant : l'eval_loss suit une descente régulière et cohérente ($0.983 \rightarrow 0.949 \rightarrow 0.950 \rightarrow 0.963$), presque identique à v2 — mais le BLEU est instable et incohérent : 60, 51, 71, 71, puis 57 à l'arrêt. Ce n'est pas le modèle qui est instable, c'est la **métrique de mesure qui est corrompue par la fuite**.

La fuite sémantique ne crée pas un avantage uniforme à chaque époque : à chaque époque, le modèle mémorise certaines structures de phrases du train set à des degrés différents. Selon quelles structures chevauchent les phrases du jeu de validation à *ce moment précis*, le BLEU monte ou descend.

La chute à l'époque 2 ($60 \rightarrow 51$) est révélatrice : le modèle commence à généraliser davantage, donc il mémorise moins parfaitement les phrases contaminées. À l'époque 3, il mémorise d'autres structures — le BLEU remonte à 71. L'early stopping à l'époque 5 (57.42) est notable : le modèle arrêté était probablement le moins overfitté des cinq.

En v2, une fois la fuite supprimée, loss et BLEU racontent la même histoire : convergence réelle à l'époque 2, puis plateau stable. L'image suivante illustre cela.

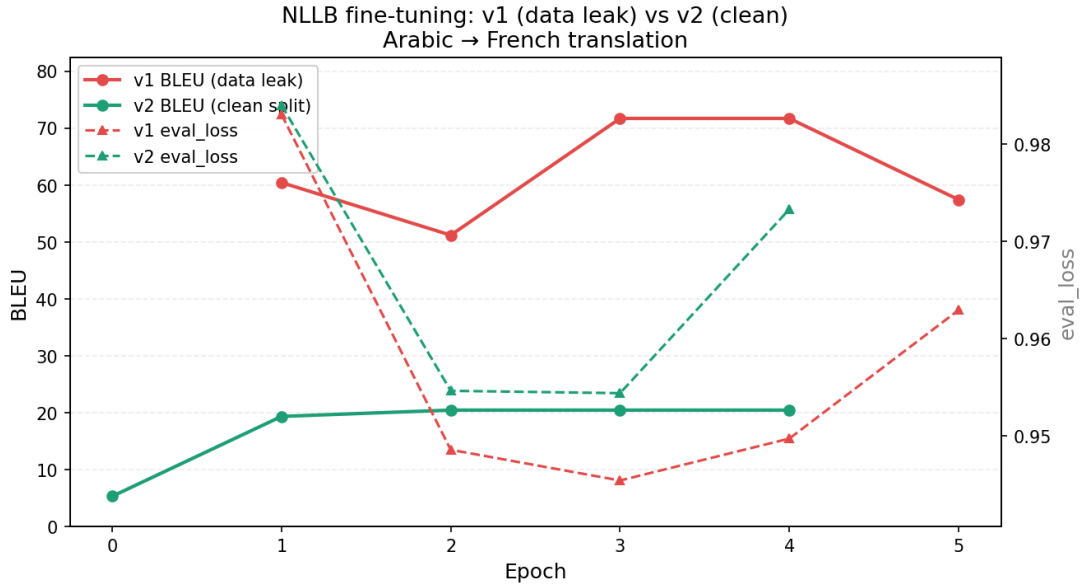


FIGURE 2 – Comparaison v1 (fuite sémantique) vs v2 (split corrigé) — courbes BLEU et eval_loss

10. Expérience de Triangulation Sémantique

10.1 Motivation et Intuition

Dans cette expérience, nous avons voulu tester le modèle `paraphrase-multilingual-mpnet-base-v2`, qui produit des embeddings de texte. L'objectif était de vérifier si ces embeddings sont véritablement **language-agnostiques** — c'est-à-dire si les relations géométriques entre phrases de même sens sont préservées d'une langue à l'autre. Si c'est le cas, on devrait pouvoir exploiter la structure de l'espace français pour naviguer dans l'espace d'une autre langue.

Pour explorer cette intuition, deux datasets parmi ceux déjà collectés pour le fine-tuning de NLLB ont été sélectionnés : `egyptian_dialogue_with_references.csv` pour l'arabe égyptien, et `test_ewe_french.csv` pour l'Éwé. Chaque dataset fournit des paires alignées français/langue cible, et les deux ont été traités de manière indépendante. Les expériences utilisent `HEAD = 4000` paires et `MASKED_RATIO = 0.7`.

10.2 Algorithme

L'algorithme appliqué sur chacun des deux datasets suit les étapes suivantes :

1. Générer `fr_embeddings` et `ar_embeddings` (ou `ewe_embeddings`) pour toutes les paires via `embed_model.encode()`
2. Calculer `fr_dist_matrix` — la matrice de distances cosinus entre tous les vecteurs français
3. Partitionner aléatoirement les indices en `masked_indices` (70% des paires, dont on cache la langue cible) et `known_indices` (les 30% restants servant de références)
4. Pour chaque `target_idx` dans `masked_indices` :
 - (a) Identifier les n entrées de `known_indices` dont le vecteur français est le plus proche de `fr_embeddings[target_idx]` dans `fr_dist_matrix`
 - (b) Pour chaque voisin `neighbor_idx`, calculer le vecteur de déplacement et la prédiction :
$$\text{embed_shift} = \text{fr}[\text{target}] - \text{fr}[\text{neighbor}] \Rightarrow \text{pred} = \text{ar}[\text{neighbor}] + \text{embed_shift}$$
 - (c) Moyenner les n vecteurs prédits en un `final_prediction`
 - (d) Évaluer :

$$\text{truth_sim} = \text{cosine_similarity}(\text{final_prediction}, \text{ar}[\text{target}])$$

L'expérience est répétée pour `n_values = [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]`.

10.3 Setup Expérimental

Paramètre	Valeur
Modèle	<code>sentence-transformers/paraphrase-multilingual-mpnet-base-v2</code>
Dimension	768
Langues couvertes	50+ (dont français, arabe, éwé)
Métrique	<code>cosine_similarity(final_prediction, ar_embeddings[target_idx])</code>
Valeurs de n	[2, 4, 8, 16, 32, 64, 128, 256, 512, 1024] — progression géométrique
Configurations	2 langues cibles $\times$ HEAD $\in \{200, 4000\} \times$ MASKED_RATIO $\in \{0.3, 0.7\}$

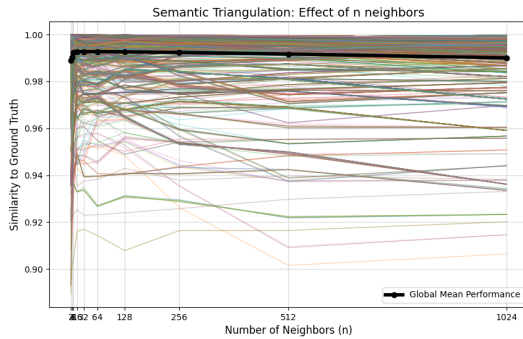
TABLE 24 – Setup expérimental — Triangulation sémantique

10.4 Résultats

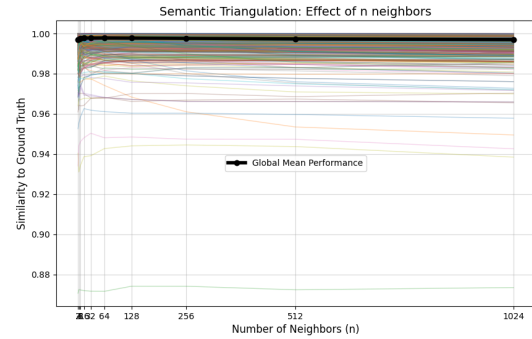
Expérience	Masquage	n optimal	truth_sim max
Arabe, HEAD=200	~50%	16–32	0.9926
Arabe, HEAD=4000	30%	64–128	0.9928
Arabe, HEAD=4000	70%	16–32	0.9927
Éwé, HEAD=4000	70%	32–64	0.9978

TABLE 25 – Résultats de triangulation — similarité cosinus maximale par configuration

Les résultats sont remarquablement stables : passer de 30% à 70% de masquage sur l'arabe ne dégrade la similarité que de 0.0001. Sur toutes les configurations, la courbe `global_history` suit le même profil : montée rapide jusqu'à $n \approx 32$, plateau stable, puis légère dégradation au-delà de $n \approx 256$ lorsque les voisins inclus deviennent sémantiquement trop éloignés.



(a) Arabe — HEAD=4000, masquage 70%



(b) Éwé — HEAD=4000, masquage 70%

FIGURE 3 – Triangulation sémantique : évolution de truth_sim vs n . Lignes colorées = paires individuelles ; ligne noire = moyenne globale.

10.5 Analyse et Interprétation

10.5.1 Robustesse au masquage

Diviser le pool de voisins connus de 2800 à 1200 paires (facteur $2.3\times$) ne dégrade la similarité que de 0.0001. La méthode n'a pas besoin d'un grand nombre de références pour fonctionner — une centaine de paires connues suffisent à obtenir des résultats stables.

10.5.2 Comportement de la courbe moyenne

Sur toutes les expériences, la courbe `global_history` suit le même profil :

- **Montée rapide** de $n = 2$ à $n \approx 32$: moyenner plusieurs voisins réduit le bruit des prédictions individuelles.
- **Plateau stable** entre $n \approx 32$ et $n \approx 128$: zone optimale.
- **Légère baisse** au-delà : les voisins inclus sont trop éloignés sémantiquement et dégradent la prédiction.

10.5.3 Ce que prouve cette expérience

Pour interpréter ces résultats, il faut d'abord poser un point de référence : la similarité cosinus mesurée directement entre le vecteur français et le vecteur éwé d'une même paire est en moyenne de l'ordre de 30%. Les deux langues occupent donc des régions bien distinctes de l'espace d'embedding — le modèle ne trivialise pas le problème en projetant toutes les langues au même endroit.

C'est ce qui rend les scores de triangulation significatifs. Obtenir 99.27% de similarité cosinus sur l'arabe et 99.78% sur l'Éwé en partant uniquement de l'espace français n'est pas un artefact de proximité — la similarité directe français/éwé de 30% le confirme. Ce qui est préservé entre langues n'est pas la position des phrases dans l'espace, mais leurs **relations géométriques** : si deux phrases françaises sont séparées par un vecteur $\mathbf{v}$, leurs traductions éwé sont séparées par approximativement le même vecteur $\mathbf{v}$. Cette propriété tient sur 70% des paires reconstruites depuis seulement 30% d'ancres connues, y compris pour l'éwé — une langue à très faibles ressources, de famille Niger-Congo, structurellement éloignée du français.

10.6 Limite

L'objectif de départ était plus large : après fusion des datasets sur la colonne française commune, le dataset résultant contient des triplets (Éwé, Français, Arabe) incomplets — chaque ligne manque soit de sa traduction arabe, soit de sa traduction éwé. L'idée était d'utiliser la triangulation pour reconstruire les vecteurs manquants, puis de les décoder en texte pour compléter le dataset.

La triangulation résout la première partie du problème : on sait localiser le vecteur manquant dans l'espace d'embedding avec plus de 99% de similarité cosinus. Mais `paraphrase-multilingual-mpnet-base-v2` est un encodeur pur — il n'existe pas de décodeur pour remonter d'un vecteur vers le texte correspondant.

Le problème est le suivant : on sait avec précision où se trouve le vecteur du texte manquant dans l'espace — mais on ne peut pas le lire.

11. Bridge MPNet → ST5 : Vers un Décodage des Embeddings

11.1 Motivation

Face à ce blocage, une piste a été explorée : convertir les représentations internes de MPNet vers celles d'un modèle capable de décoder en texte.

Le modèle ST5 (`t5-base`) est un encodeur-décodeur entraîné sur l'anglais, capable de générer du texte à partir de ses propres états internes. L'idée est d'entraîner un réseau intermédiaire — un `CrossModelBridge` — à transformer la matrice d'états cachés produite par MPNet (`[batch, seq_len, 768]`) en une matrice équivalente dans l'espace interne de ST5.

11.2 Pipeline Cible

Si cela fonctionne, la chaîne complète devient :

$$\text{texte arabe / éwé} \xrightarrow{\text{MPNet}} \text{embedding} \xrightarrow{\text{Bridge}} \text{espace ST5} \xrightarrow{\text{décodeur T5}} \text{texte anglais}$$

Ce pipeline permet de créer des **paires synthétiques** (texte cible, anglais) à partir de n'importe quelle ressource textuelle dans la langue cible — même non alignée. Ces paires peuvent ensuite servir à fine-tuner NLLB sur ces langues, jusqu'à pousser le BLEU score significativement plus haut (cible : ~ 40).

Un NLLB ainsi fine-tuné devient suffisamment fiable pour générer les voix manquantes dans le dataset mergé sur le français. La qualité de ces traductions peut être validée de manière intrinsèque : en comparant l'embedding MPNet de la phrase produite par NLLB fine-tuné avec le vecteur triangulé d'origine. Si les deux sont proches dans l'espace MPNet, la traduction est sémantiquement fidèle — ce qui boucle avec l'expérience de triangulation.

11.3 Architecture du Bridge

Le `CrossModelBridge` implémenté dans `mpnet_hidden_to_matrix_to_st5_hidden_matrix_v1.py` prend en entrée `mpnet_model(**m_in).last_hidden_state` et produit une matrice de dimension `[batch, SEQ_LEN=36, 768]`. Il est composé de :

- Une **attention croisée** (`MultiheadAttention`, 12 têtes) avec des queries apprenables
- Un **TransformerEncoder** à 6 couches
- Une **LayerNorm** finale

La loss est une **MSE masquée** — on n’entraîne que sur les tokens réels, les tokens de padding étant exclus via `t_in['attention_mask']`.

11.4 Résultats

L’entraînement sur `egyptian_dialogue_with_references.csv` converge sur 163 époques (early stopping, patience=20) avec un scheduler `ReduceLROnPlateau` divisant le LR par 2 à chaque plateau.

Époque	Val Masked Loss	LR
1	0.880	1.0×10^{-4}
50	0.224	1.0×10^{-4}
100	0.074	1.0×10^{-4}
135	0.041	1.0×10^{-4}
143 (best)	0.0405	5.0×10^{-5}
163 (early stop)	0.041	7.8×10^{-7}

TABLE 26 – Convergence du bridge — val loss au fil des époques

Le test final de décodage montre que le bridge capte vaguement le sens :

Original	Décodé
<i>At first, I didn't believe it either.</i>	<i>I didn't believe it.</i>
<i>Send me your news.</i>	<i>Send me a message.</i>
<i>It's the same as I told you about lucid dreams.</i>	<i>It's like a dream come true.</i>
<i>The phone is out of service.</i>	<i>This is the first time I've ever had a problem with my phone.</i>
<i>The custodian did not grant us permission.</i>	<i>He did not know that we were going to have to pay for it.</i>

TABLE 27 – Exemples de décodage — bridge MPNet → ST5 (v1)

11.5 Conclusion et Perspectives

Ces résultats sont encourageants : le bridge arrive à capturer une partie du sens des phrases, ce qui montre que la conversion entre les deux espaces de représentation n’est pas hors de portée. Cependant, le plancher à ~ 0.04 de val loss — atteint dès l’époque 143 et impossible à franchir malgré 20 époques supplémentaires de patience et une réduction du LR ($10^{-4} \rightarrow 7.8 \times 10^{-7}$) — suggère que l’approche v1 ne converge pas vers une précision suffisante pour générer des données d’entraînement fiables. Ce plancher semble indiquer une limite structurelle, probablement liée à la distance entre les deux espaces de représentation.

Pour aller plus loin, il faudra explorer des approches radicalement différentes. Dans cette direction, le projet **Vec2Text** de l’Université d’Arizona constitue une piste prometteuse : il s’attaque directement au problème de la génération de texte à partir d’embeddings denses et arrive à reconstruire des phrases avec une fidélité remarquable. Adapter cette approche aux embeddings MPNet multilingues — notamment pour l’arabe et l’ewé — serait une voie naturelle à explorer dans la suite de ce travail.

A. Ressources du Projet

- **Code source (GitHub)** : <https://github.com/steve-sanogo/medbridge-speech-stack>
- **Modèles et datasets (Hugging Face)** : <https://huggingface.co/medbridge-ai>